# Examen Final

L'examen dure 2 heures et comporte 5 exercices. Les documents sont autorisés, mais limités à 5 feuilles recto-verso par personne. Aucun appareil numérique n'est autorisé, donc, de façon non-exhaustive : pas d'ordinateur, pas de téléphone, pas de montre connectée, etc. Pas de calculatrice non plus.

## Exercice 1 (Degrés sortants et entrants)

Étant donné un graphe orienté $\vec{G}$ on définit le graphe non-orienté $G$ de la façon suivante : $V(G) = V(\vec{G})$ et deux sommets $x$ et $y$ sont adjacents dans $G$ si et seulement $xy$ ou $yx$ est un arc de $\vec{G}$.

Expliquez comment calculer la liste d'adjacence de $G$ étant donné celle de $\vec{G}$. Pas besoin de donner un pseudo-code, mais justifiez la complexité.

## Exercice 2 (Dijkstra et arcs de poids négatif)

Un étudiant étourdi a exécuté l'algorithme de Dijkstra sur le graphe de la Figure 1 avec comme source $s$, sans s'apercevoir que celui-ci avait des arcs de poids négatif. Exécuter l'algo sur le graphe (avec le tableau de sommets, comme vu en cours). Donner un sommet pour lequel l'algorithme ne renvoie pas la bonne distance depuis $s$.

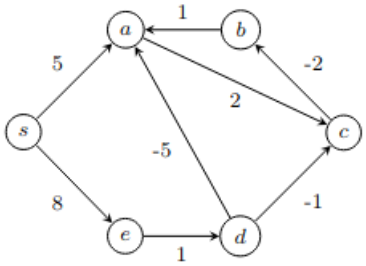



FIGURE 1 – Graphe de l'exercice 2

## Exercice 3 (Fermeture transitive)

Dans cet exercice on considère des graphes dirigés tel que, pour toute paire de sommets $x, y$, il y a au plus un arc allant de $x$ vers $y$, mais il peut y avoir un arc de $x$ vers $y$ et un autre de $y$ vers $x$.

Étant donné un graphe dirigé $G = (V, A)$ où $V = \{v_1, v_2, \ldots, v_n\}$, la fermeture transitive de $G$ est le graphe $G^* = (V, A^*)$, où $v_i v_j \in A^*$ si et seulement si il existe un chemin de $v_i$ à $v_j$ dans $G$.

**Question 1** Dessiner la fermeture transitive du graphe orienté suivant : $V = \{v_1, v_2, v_3, v_4, v_5\}$, $A = \{v_1v_2, v_1v_5, v_2v_3, v_3v_4, v_5v_4\}$.

**Question 2** Combien d'arcs a la fermeture transitive du chemin sur $n$ sommets $P_n$ (c'est à dire le graphe dirigé sur les sommets $v_1, \ldots, v_n$ avec les arcs $\{v_1v_2, v_2v_3, \ldots, v_{n-1}v_n\}$) ?

**Question 3** Quelle est la fermeture transitive d'un graphe dirigé $G$ fortement connexe [1] ?

**Question 4** Combien d'arcs a la fermeture transitive du cycle sur $n$ sommets $C_n$ (c'est à dire le graphe dirigé sur les sommets $v_1, \ldots, v_n$ avec les arcs $\{v_1v_2, v_2v_3, \ldots, v_{n-1}v_n, v_nv_1\}$) ?

On définit, pour $i, j, k \in \{1, \ldots, n\} \times \{1, \ldots, n\} \times \{0, \ldots, n\}$, $TR(i, j, k)$ comme étant égal à 1 s'il existe un chemin de $v_i$ vers $v_j$ n'utilisant que des sommets de $\{v_i, v_j\} \cup \{v_1, \ldots, v_k\}$.

**Question 5** Donnez un algorithme qui calcule la fermeture transitive d'un graphe $G$ en utilisant de la programmation dynamique. On pourra utiliser les sous-problèmes TR définit ci-dessus.

On vous demande de : 1- Traiter les cas de bases, 2- Expliciter la formule de récurrence, 3- Donner un pseudo-code (sans oublier les cas de bases).

**ATTENTION :** Il est interdit d'utiliser des algorithmes étudiés en cours, c'est à dire qu'on ne peut pas dire « j'exécute Dijkstra » ou autre (on peut en revanche s'inspirer des idées vues dans des algos).

## Exercice 4 (Tas-max)

**Question 1** Dessinez tous les tas-max contenant les éléments $\{1, 2, 3\}$, puis les éléments $\{1, 2, 3, 4\}$.

**Question 2** Insérer (en utilisant la procédure vue en cours) un élément de clé 17 dans le tas-max de la Figure 2. Répondre en représentant le nouveau tas.

**Question 3** En repartant du tas initial, retirer (en utilisant la procédure vue en cours) l'élément de clé maximum. Répondre en représentant le nouveau tas.

**Question 4** On a vu dans le cours qu'un tas-max peut-être implémenté dans un tableau. Représentez le tableau suivant comme un arbre. $[23, 17, 14, 6, 13, 10, 1, 5, 7, 12]$. Est-ce un tas-max ?

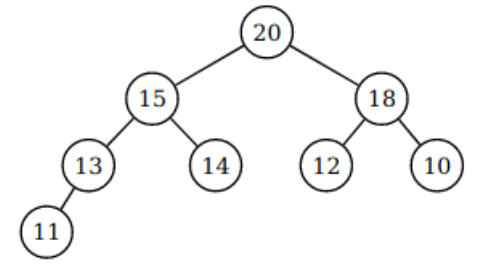



FIGURE 2 – Tas-max de l'exercice 4

1. On rappelle qu'un graphe est fortement connexe si pour toute paire de sommets $u, v$, il existe un chemin de $u$ vers $v$.

## Exercice 5 (Labyrinthe – Python)

Vous êtes dans un labyrinthe. Vous devez retrouver *le plus court chemin* du point de départ `S` jusqu'à la sortie `E`. Pour ce faire, on veut déterminer la séquence de déplacements à faire pour rejoindre la sortie. Le labyrinthe est représenté comme une grille de caractères, où chaque case de la grille peut être :

- `#` : un mur (non franchissable),
- `.` : une case libre donc franchissable,
- `S` : le point de départ (start),
- `E` : la sortie (end).

On peut se déplacer **uniquement dans les 4 directions orthogonales** : `U` = haut (up), `D` = bas (down), `L` = gauche (left), `R` = droite (right). Voici un exemple de labyrinthe suivant cette représentation :

```
exemple_labyrinthe = [
    "######",
    "#.S..#",
    "#.##.#",
    "#....#",
    "###E##"
]
```

Une façon de s'échapper de ce labyrinthe en faisant un minimum de déplacements serait de faire les déplacements `"LDDRRD"`. Notez que pour certains labyrinthes il est possible d'avoir plusieurs chemins ayant une longueur minimale : retourner l'un d'entre eux est suffisant (dans l'exemple, `"RRDDLD"` marche également).

**Question 1** Pourquoi l'utilisation de l'algorithme BFS est particulièrement adaptée pour ce problème du plus court chemin ? Expliquer.

**Question 2** Écrire une fonction Python `trouver_debut_sortie(labyrinthe)` prenant en paramètre un labyrinthe possédant la structure décrite plus haut, et retournant deux tuples Python correspondant à la position (les coordonnées) du départ et de la sortie dans le labyrinthe. *Rappels/Indice : Pour retourner deux éléments a et b, on écrit* `return a, b`*. On rappelle aussi qu'en Python,* ***une chaîne de caractères peut être vue comme une liste de caractères****. Ainsi, le labyrinthe n'est rien d'autre qu'une liste de listes...*

**Question 3** Compléter le code Python de la fonction `plus_court_chemin(labyrinthe)`, partout où des champs « `____` » apparaissent. Quelques indications :

- **Vous devez répondre dans votre copie et non sur le sujet** : vous pouvez pour ce faire écrire le numéro de la ligne suivi de la ligne complétée.
- Le nombre de « `_` » est choisi aléatoirement et ne reflète donc pas le nombre de caractères attendu.
- Des précisions sont données sous la forme de commentaires Python pour aider à la compréhension.

```python
from collections import deque

def plus_court_chemin(labyrinthe):
    n, m = len(labyrinthe), len(labyrinthe[0])
    debut, sortie = trouver_debut_sortie(labyrinthe)

    deja_visitees = [[False]*m for _ in range(n)] # Structure de donnees pour garder trace des cases deja visitees
    deplacements = {}  # Dictionnaire gardant trace des differents deplacements realises : (x, y) -> (px, py, direction)

    # On initialise la file et on visite la case de depart
    file_ = deque() # file est un mot reserve en Python, on utilise file_
    file_.append(debut)
    deja_visitees[debut[0]][debut[1]] = True

    # Les deplacements possibles sont les directions orthogonales
    directions = [(-1,0,'U'), (1,0,'D'), (0,-1,'L'), (0,1,'R')]

    while file_:
        x, y = file_.popleft()
        if (x, y) == sortie:
            break
        for dx, dy, lettre in directions:
            nx, ny = x + dx, y + dy
            if _________________________: # Condition(s) pour ne pas chercher en dehors du tableau
                if _________________________: # Condition(s) pour pouvoir visiter la case (nx, ny)
                    deja_visitees[___][___] = _____
                    file_.append(_____)
                    deplacements[(nx, ny)] = (x, y, lettre)

    # Reconstruction du chemin en partant de la sortie et en remontant jusqu'au debut
    chemin = []
    case_courante = sortie
    while case_courante != debut:
        px, py, lettre = ____________
        chemin.append(__________)
        case_courante = (px, py)

    return ''.join(reversed(chemin)) # On retourne les deplacements dans le bon ordre
```

**Question 4** Quelle est la complexité temporelle de cet algorithme ? Justifier.